



Figure 8. Inference-time guidance methods arranged by computational cost. Approximate guidance methods (e.g., $g^{\text{cov-G}}$) are lightweight but accumulate local approximation error, leading to off-manifold drift. Exact guidance methods (e.g., Guidance Matching, sample-based guidance) are exact but require substantially more compute. This work (g^{car}), which sits in the middle, aims to improve the compute-light approximate guidance by adding extra compute.

A. Extended related works

A.1. Inference-time alignment

Inference-time alignment methods for flow matching models refers to steering the generated samples toward some constraints, e.g., sampling from a distribution weighted with some objective function (Lu et al., 2023) or conditioned on class labels (Song et al., 2021), and all constraints can be framed as reward functions $r : \mathbb{R}^d \rightarrow \mathbb{R}$. Approaches that Inference-time reward alignment for flow models can be divided into three broad paradigms:

Inference-time guidance. Inference-time guidance addresses the reward-tilted sampling problem by adding a guidance vector field $g_t(x_t)$ to the pretrained velocity $v_\theta(x_t, t)$ during ODE integration, leaving the pretrained model unchanged. A unified theoretical framework for this family was established by Feng et al. (2025), who derive the exact guidance vector field for general flow matching:

$$g_t(x_t) = \mathbb{E}_{z \sim p(z|x_t)} \left[\left(\frac{e^{r(x_1)}}{Z_t(x_t)} - 1 \right) v_{t|z}(x_t|z) \right],$$

where $Z_t(x_t) = \mathbb{E}_{z \sim p(z|x_t)} [e^{r(x_1)}]$ is an intractable normalising constant. Methods in this family differ in how they approximate g_t .

Approximate guidance replaces the intractable posterior average with a point estimate via the Tweedie/ $\hat{x}_1$ -predictor, adapting well-studied diffusion guidance methods including DPS (Chung et al., 2023), IIGDM (Song et al., 2023a), and LGD (Song et al., 2023b) to the flow matching setting; Feng et al. (2025) unify these under the flow-matching extension $g^{\text{cov-G}}$. These methods are computationally lightweight but incur an approximation error, as we show in Section 4 and as also discussed in Feng et al. (2025).

Exact guidance methods avoid this bias at the cost of additional computation. On the training-free side, Monte Carlo guidance (g^{MC} , Feng et al., 2025) estimates g_t by drawing N samples from the prior $p(z)$ and self-normalising; it is asymptotically exact but suffers from high variance in high-dimensional spaces. GLASS Flows (Holderrieth et al., 2026) combine exact guidance with Sequential Monte Carlo (SMC) via Feynman-Kac Steering, treating ODE sampling as a search tree and using stochastic transitions to explore branches (i.e., replacing the inefficient SDE steps required by prior SMC methods with ODE-level efficiency). On the training-based side, Guidance Matching (Feng et al., 2025) learns a network g_ψ to directly approximate g_t via tractable surrogate losses, which require only reward-labelled samples from the ground-truth data distribution.

Optimization-based controlled generation. A second paradigm frames controlled generation as a direct optimization problem: given a differentiable objective (cost or reward), one searches for an initial noise, latent trajectory, or auxiliary variable that, after running the generative ODE/SDE, produces a sample of high reward. Representative methods include diffusion models as plug-and-play priors (Graikos et al., 2022), compositional generation via MCMC on the energy landscape (Du et al., 2023), latent optimization approaches (Mardani et al., 2023; Samuel et al., 2023), and methods that *differentiate through* the entire sampling ODE to back-propagate reward gradients into the input space, including D-Flow (Ben-Hamu et al., 2024), FlowGrad (Liu et al., 2023c), and source-guided flow matching (Wang et al., 2025). Those methods target a fundamentally different objective from the guidance framework we adopt: instead of considering a sampling problem from the reward-tilted distribution $p'_1(x_1) \propto p_1^{\text{base}}(x_1) e^{r(x_1)}$ in a distributional sense, they consider an optimization problem and seek a single high-reward mode.

Reward fine-tuning. A third paradigm modifies the pretrained model weights to maximise the reward, via stochastic optimal control (Domingo-Enrich et al., 2025), optimal control (Zhang et al., 2025b), DPO (Wallace et al., 2024), GRPO (Xue et al., 2025; Liu et al., 2025a) or other reinforcement learning approaches. A third paradigm modifies the pretrained model weights to maximize the reward, based on GRPO (Xue et al., 2025; Liu et al., 2025a), stochastic optimal control (Domingo-Enrich et al., 2025), optimal control, DPO (Wallace et al., 2024) or other reinforcement learning approaches. They differ in how this optimal control problem is solved, for example, VGG-Flow (Zhang et al., 2025b) finetunes the corrected velocity via a reward-importance-weighted flow matching loss, whereas Adjoint Matching (Domingo-Enrich et al., 2025) back-propagates through the entire ODE trajectory using the continuous adjoint equations. Many fine-tuning methods require DDPM/SDE sampling for exploration, during training (Xue et al., 2025; Liu et al., 2025a; Domingo-Enrich et al., 2025), which is significantly less efficient than ODE sampling and couples the method to a specific reward at training time; adapting to a new reward requires retraining from scratch. We instead focus on exploring how to best leverage the pretrained flow model at inference time, without any fine-tuning.

A.2. Value gradient guidance

A related line of work defines the guidance signal as the gradient of a learned value function $g(x_t, t) \triangleq \nabla_{x_t} V(x_t, t)$, where $V(x_t, t) \approx \mathbb{E}[r(x_1) \mid x_t]$ summarizes the expected terminal reward. VGG-Flow (Liu et al., 2025b) instantiates this idea by identifying the gap between the pretrained and fine-tuned velocity fields as the value gradient — it co-trains a value gradient network via an HJB consistency loss alongside the fine-tuned velocity, using each to supervise the other. As a fine-tuning method, VGG-Flow couples the model to a single fixed reward at training time and does not address off-manifold drift: the learned value gradient is a general, unconstrained vector field with no mechanism to correct trajectories that have already deviated from the data manifold. In contrast, our g^{car} targets the failure mode of inference-time guidance methods, where off-manifold drift arises specifically in regions of gradient conflict — when multiple reward gradients point in opposing directions, their sum vanishes or points into low-density regions, trapping trajectories away from the data manifold. Rather than fine-tuning the model, g^{car} learns a lightweight residual correction that is selectively activated in these conflict regions at inference time, redirecting trajectories back onto the data manifold without modifying pretrained weights, and can be applied on top of any heuristic approximate guidance.